

Diabetes Prediction using Machine Learning

End-to-End Classification Pipeline — Assignment Report

Dataset: Pima Indians Diabetes Dataset (768 records, 9 attributes)

Algorithms: Logistic Regression, Decision Tree, Random Forest

Tools: Python, Scikit-learn, Pandas, Matplotlib, Seaborn

Table of Contents

1. Data Preparation

- 1.1 Dataset Overview
- 1.2 Exploratory Data Analysis
- 1.3 Data Preprocessing
- 1.4 Feature-Target Separation and Train-Test Split

2. Machine Learning Model Building

- 2.1 Algorithms Implemented
- 2.2 Key Model Parameters

3. Model Training

- 3.1 Training Results
- 3.2 Overfitting / Underfitting Observations

4. Model Evaluation

- 4.1 Performance Metrics Comparison
- 4.2 Confusion Matrix Analysis
- 4.3 Training vs Testing Performance

5. Model Performance Analysis

- 5.1 Key Findings
- 5.2 Model Limitations
- 5.3 Recommendations for Improvement

1. Data Preparation

1.1 Dataset Overview

The dataset used is the **Pima Indians Diabetes Dataset**, containing 768 patient records with 9 attributes. The target variable is **Outcome** (0 = No Diabetes, 1 = Diabetes).

Dataset Shape: 768 rows × 9 columns

Column	Data Type	Description
Pregnancies	int64	Number of times pregnant
Glucose	int64	Plasma glucose concentration
BloodPressure	int64	Diastolic blood pressure (mm Hg)
SkinThickness	int64	Triceps skin fold thickness (mm)
Insulin	int64	Serum insulin level (mu U/ml)
BMI	float64	Body Mass Index
DiabetesPedigreeFunction	float64	Diabetes pedigree score
Age	int64	Age of the patient
Outcome	int64	Target (0 = No Diabetes, 1 = Diabetes)

1.2 Exploratory Data Analysis

Statistical Summary: The dataset contains no explicit null values. However, several columns have biologically implausible zero values that represent missing data.

Statistic	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
Count	768	768	768	768	768	768
Mean	3.85	120.89	69.11	20.54	79.80	31.99
Std	3.37	31.97	19.36	15.95	115.24	7.88
Min	0.00	0.00	0.00	0.00	0.00	0.00
Max	17.00	199.00	122.00	99.00	846.00	67.10

Target Variable Distribution: The dataset is imbalanced with 500 records (65.1%) for No Diabetes and 268 records (34.9%) for Diabetes.

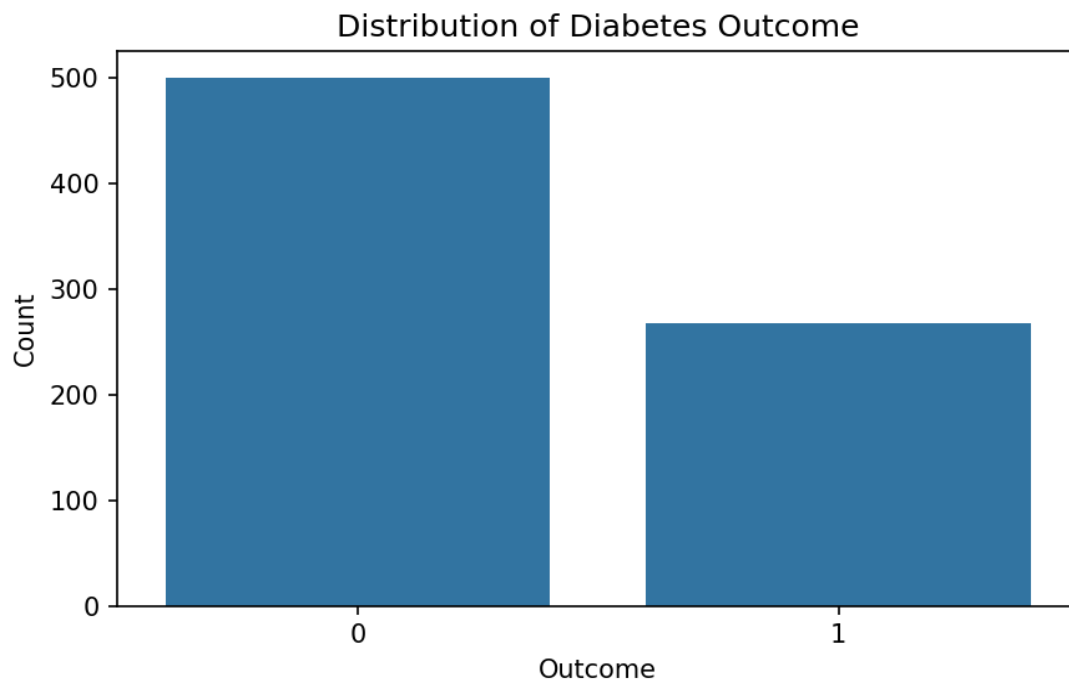


Figure 1: Distribution of Diabetes Outcome (0 = No Diabetes, 1 = Diabetes)

1.3 Data Preprocessing

Several columns contain zero values that are biologically impossible and represent missing data. The following zero-value counts were identified:

Column	Zero Values
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11

Preprocessing Steps Performed:

- Zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI were replaced with NaN.
- Missing values were imputed using the **median** strategy via Scikit-learn's SimpleImputer.
- Numerical features were standardized using **StandardScaler** for Logistic Regression (Decision Tree and Random Forest do not require scaling).
- No categorical encoding was required as all features are numerical.

1.4 Feature-Target Separation and Train-Test Split

Features (X): Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age

Target (y): Outcome

Split Configuration: 80% training, 20% testing, stratified by target variable, random_state=42.

Set	Samples	Features
Training	614	8
Testing	154	8

2. Machine Learning Model Building

2.1 Algorithms Implemented

Three classification algorithms were implemented and compared:

- 1. Logistic Regression** — A linear model that predicts the probability of diabetes by fitting a logistic function to the input features. It is suitable for binary classification problems and provides interpretable coefficients. It is trained on standardized features to ensure convergence.
- 2. Decision Tree Classifier** — A non-linear, tree-based model that recursively splits data based on feature thresholds to maximize class separation. It captures non-linear relationships in the data and does not require feature scaling. The `max_depth` parameter is set to 5 to prevent overfitting.
- 3. Random Forest Classifier** — An ensemble method that builds 100 decision trees on bootstrapped samples and aggregates their predictions via majority voting. It reduces variance compared to a single decision tree and provides feature importance scores.

2.2 Key Model Parameters

Model	Key Parameters
Logistic Regression	<code>max_iter=1000, random_state=42, solver=lbfgs</code>
Decision Tree	<code>max_depth=5, random_state=42</code>
Random Forest	<code>n_estimators=100, random_state=42</code>

3. Model Training

3.1 Training Results

All three models were trained on the 614-sample training set. The following training accuracies were observed:

Model	Training Accuracy
Logistic Regression	79.64%
Decision Tree	81.43%
Random Forest	100.00%

3.2 Overfitting / Underfitting Observations

Logistic Regression: Training accuracy of 79.64% and testing accuracy of 70.78% show a moderate gap (~9%). The model slightly underfits the data, likely due to its linear decision boundary not capturing non-linear patterns in the features.

Decision Tree: Training accuracy of 81.43% and testing accuracy of 75.97% show a small gap (~5.5%). The `max_depth=5` constraint effectively controls overfitting, resulting in a well-balanced model.

Random Forest: Training accuracy of 100% and testing accuracy of 77.92% reveal a significant gap (~22%). This indicates **overfitting** — the ensemble has memorized the training data. Despite this, it still achieves the highest test accuracy among the three models.

4. Model Evaluation

4.1 Performance Metrics Comparison

The following table summarizes the test set performance of all three models:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	70.78%	60.00%	50.00%	54.55%
Decision Tree	75.97%	63.93%	72.22%	67.83%
Random Forest	77.92%	72.73%	59.26%	65.31%

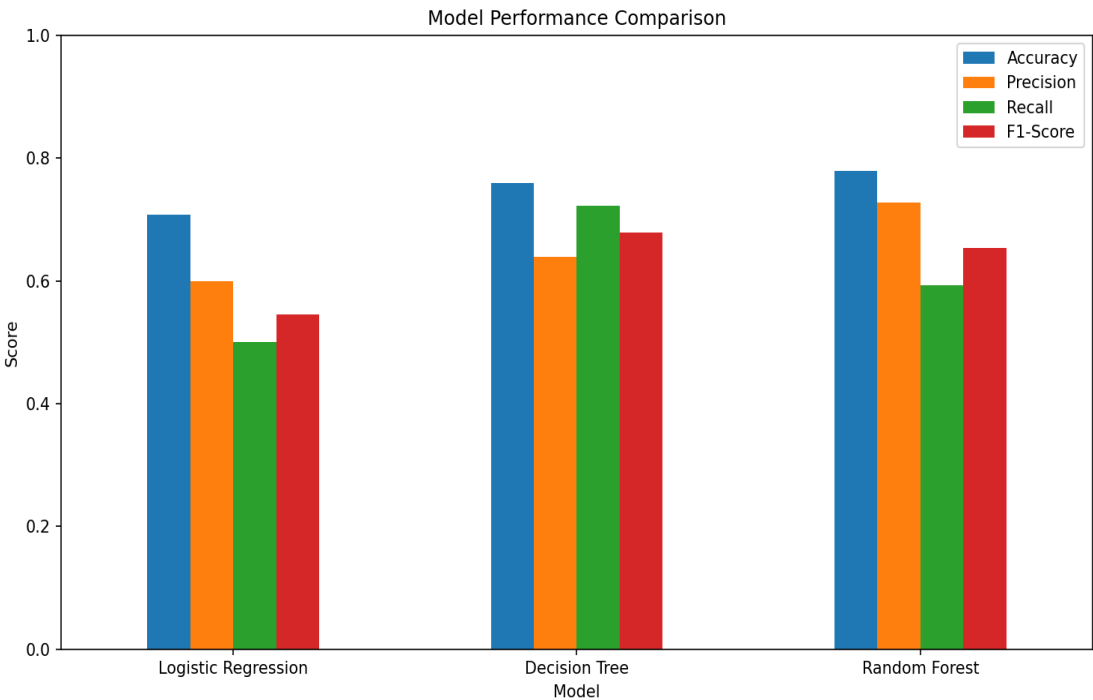


Figure 2: Model Performance Comparison across Accuracy, Precision, Recall, and F1-Score

4.2 Confusion Matrix Analysis (Random Forest)

The confusion matrix for the Random Forest model on the test set is shown below:

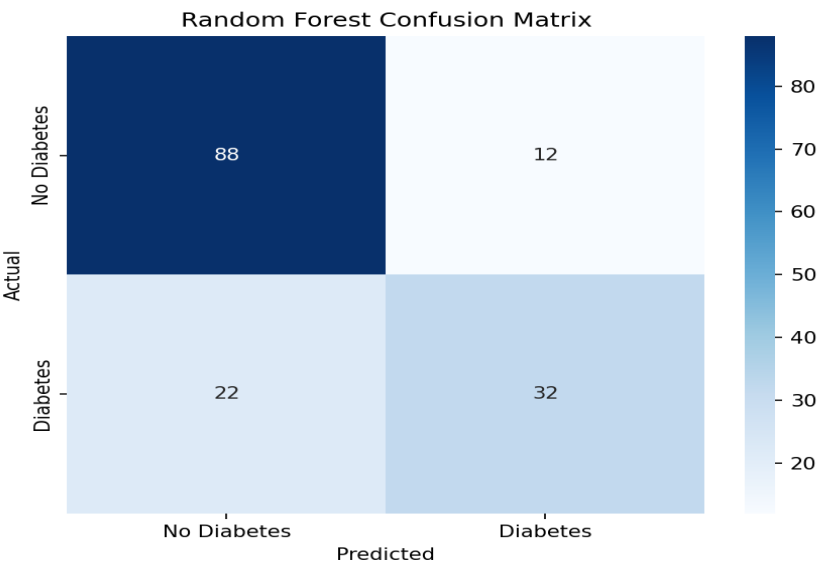


Figure 3: Random Forest Confusion Matrix on Test Set

Metric	Value	Interpretation
True Negatives (TN)	88	Correctly predicted No Diabetes
False Positives (FP)	12	Incorrectly predicted Diabetes
False Negatives (FN)	22	Missed Diabetes cases
True Positives (TP)	32	Correctly predicted Diabetes

The model correctly identifies 88 out of 100 non-diabetic patients and 32 out of 54 diabetic patients. The 22 false negatives are clinically significant as they represent missed diabetes diagnoses.

4.3 Training vs Testing Performance

Model	Train Accuracy	Test Accuracy	Gap
Logistic Regression	79.64%	70.78%	8.86%
Decision Tree	81.43%	75.97%	5.46%
Random Forest	100.00%	77.92%	22.08%

The Decision Tree shows the smallest train-test gap, indicating the best generalization among the three models. The Random Forest's 22% gap confirms overfitting despite its higher test accuracy.

5. Model Performance Analysis

5.1 Key Findings

Feature Importance (Random Forest): Glucose is the most influential predictor of diabetes, followed by BMI, DiabetesPedigreeFunction, and Age. SkinThickness contributes the least.

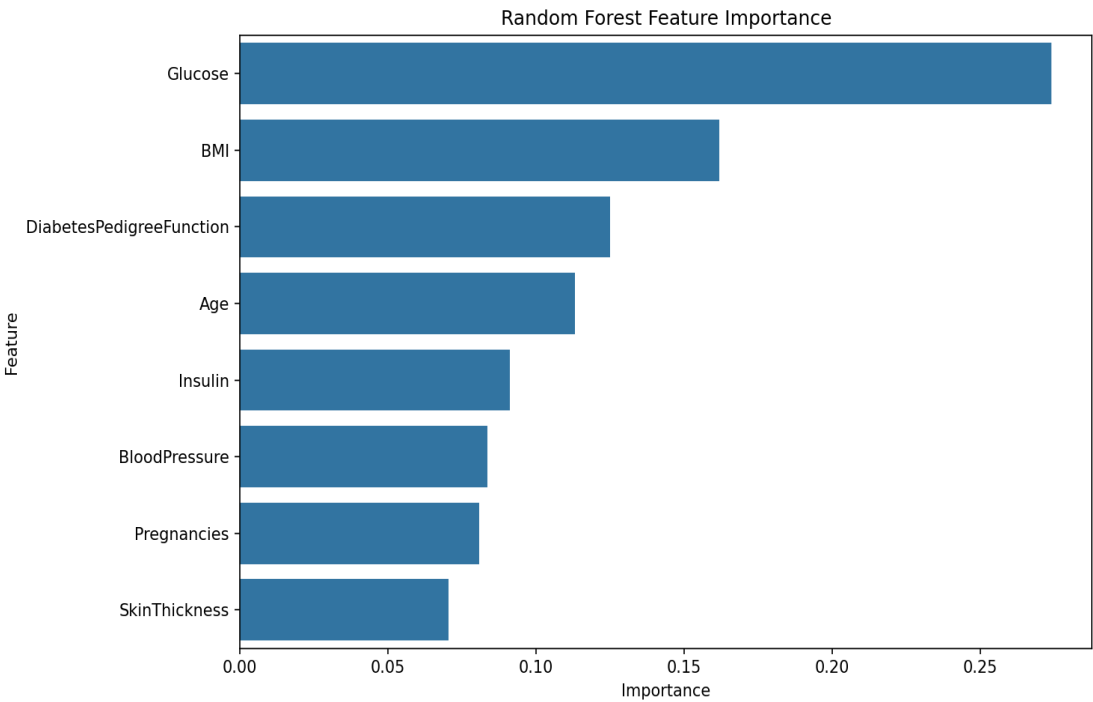


Figure 4: Random Forest Feature Importance Ranking

Best Model by F1-Score: The **Decision Tree Classifier** achieves the highest F1-Score (67.83%), providing the best balance between precision and recall. This is the most appropriate metric for diabetes prediction because both false positives (unnecessary treatment) and false negatives (missed diagnoses) carry clinical consequences.

Best Model by Accuracy: The **Random Forest Classifier** achieves the highest test accuracy (77.92%) and highest precision (72.73%), but at the cost of lower recall (59.26%).

5.2 Model Limitations

- The dataset is relatively small (768 samples) which limits model generalizability.
- Class imbalance (65% vs 35%) biases models toward the majority class (No Diabetes).
- Random Forest shows clear overfitting (100% training accuracy vs 77.92% testing accuracy).
- All models have recall below 73%, meaning a significant portion of diabetic patients are missed.
- The dataset originates from a specific demographic (Pima Indian women aged 21+), limiting broader applicability.

5.3 Recommendations for Improvement

- **Handle Class Imbalance:** Apply SMOTE (Synthetic Minority Oversampling Technique) or `class_weight='balanced'` to improve recall for the minority (Diabetes) class.
- **Hyperparameter Tuning:** Use `GridSearchCV` or `RandomizedSearchCV` to optimize model parameters (e.g., `max_depth`, `n_estimators`, `min_samples_split` for tree-based models).
- **Feature Engineering:** Create interaction features (e.g., Glucose $\times$ BMI) or polynomial features to capture non-linear relationships.
- **Try Additional Algorithms:** Evaluate Gradient Boosting (XGBoost, LightGBM) or Support Vector Machines, which may yield better performance on this dataset.
- **Cross-Validation:** Use k-fold cross-validation instead of a single train-test split for more robust performance estimates.